

API Specification & Developer Documentation

Poly-Cipher Socket Protocol (PCSP)

Version 2.0 — Extended Edition

Engineering Subsystem Contracts

Abstract

This document serves as the definitive engineering manual for the Poly-Cipher Socket Protocol (PCSP). It defines the standard C++ data structures, application programming interfaces (APIs), and memory ownership semantics required for parallel, modular development. This edition extends the original interface specification with the full Network Transport contract (REQ-NET), the Diagnostic Telemetry contract (REQ-DSP), a complete Cipher ID Registry spanning all eleven ciphers defined in the Software Requirement Specification, and the Non-Functional safety contract (REQ-NFR). Adherence to these interfaces guarantees that the network parsing loop, protocol serialization, cryptographic mathematics, console diagnostics, and error recovery can be developed, tested, and integrated independently.

Contents

1	Architectural Philosophy & Memory Semantics	3
1.1	System Overview	3
2	Network Transport Layer (REQ-NET)	3
2.1	Server Socket Contract (REQ-NET-101, REQ-NET-102)	4
2.2	Client Socket Contract (REQ-NET-103)	4
2.3	Byte-Order Normalization (REQ-NET-104)	5
3	Protocol Structures & Serialization (REQ-PRT)	5
3.1	Header Structure Definition	5
3.2	Serialization Contract	5
3.3	Server Parsing State Machine (REQ-PRT-202, REQ-PRT-203)	6
4	Diagnostic & Telemetry Contract (REQ-DSP)	7
4.1	Shared Hex-Dump Utility	7
4.2	Client-Side Telemetry (REQ-DSP-301)	7
4.3	Server-Side Telemetry (REQ-DSP-302)	7

5	The Cryptographic Subsystem API (REQ-CIP)	8
5.1	The ICipher Interface	8
5.2	Cipher ID Registry	8
5.3	The Cipher Factory & Polymorphic Dispatch	8
5.4	Implementation Example: Caesar Cipher (REQ-CIP-401)	9
5.5	Implementation Example: Vigenère Cipher (REQ-CIP-501)	10
5.6	Key Field Interpretation Notes for Remaining Ciphers	11
6	Non-Functional Requirements & Safety (REQ-NFR)	12
6.1	Memory Constraints (REQ-NFR-801)	12
6.2	Graceful Error Recovery (REQ-NFR-802)	12
7	Division of Labor & Integration	13

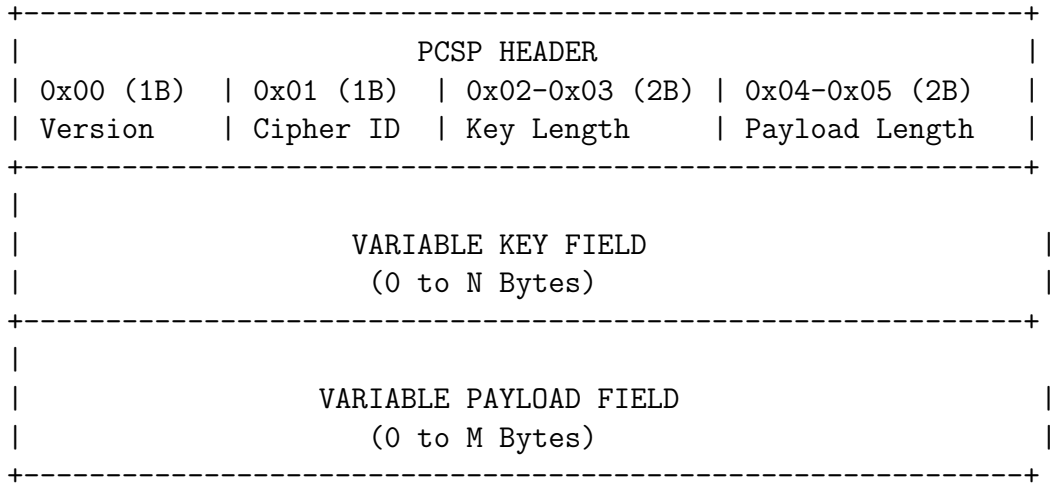
1 Architectural Philosophy & Memory Semantics

When multiple developers coordinate across a codebase—particularly one handling raw socket streams and variable-length mathematical payloads—interface boundaries must be rigid. C and C++ natively treat data streams as raw memory pointers (`uint8_t*` or `char*`), often relying on null-termination conventions. In cryptography, null bytes (`0x00`) are valid ciphertext states, rendering standard string libraries unsafe.

We eliminate this ambiguity by enforcing strict boundary conditions. No subsystem shall pass raw pointers without explicit, coupled length parameters. To satisfy the bounds-checked memory constraints mandated by REQ-NFR-801, the canonical data structure for all plaintexts, ciphertexts, and key vectors is `std::vector<uint8_t>`. This enforces Resource Acquisition Is Initialization (RAII) and automates bounding.

1.1 System Overview

PCSP implements a synchronous, stateful Client-Server architecture over standard POSIX stream sockets (`SOCK_STREAM`). The server follows an *iterative model*: it accepts one client, fully services the request, and closes the connection before returning to listen for the next. Every transmitted message is packed contiguously as a fixed 6-byte header followed by two variable-length fields:



This layout is the single source of truth shared by every subsystem in this document: the network layer moves the bytes, the protocol layer interprets the lengths, and the cryptographic layer never sees a socket at all.

2 Network Transport Layer (REQ-NET)

The transport layer is intentionally the “dumbest” subsystem in PCSP: it moves bytes and normalizes their byte order, and nothing else. A developer implementing this layer must not need to know what a cipher ID means, only how many bytes to move.

2.1 Server Socket Contract (REQ-NET-101, REQ-NET-102)

The server instantiates a passive stream socket bound to a configurable port, then services clients one at a time (Iterative Server Model): accept, process to completion, print diagnostics, close, and return to `accept()`.

```
1 #include <sys/socket.h>
2 #include <netinet/in.h>
3 #include <unistd.h>
4
5 int start_server(uint16_t port) {
6     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
7     int opt = 1;
8     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
9
10    sockaddr_in addr{};
11    addr.sin_family = AF_INET;
12    addr.sin_addr.s_addr = INADDR_ANY;
13    addr.sin_port = htons(port);
14
15    bind(server_fd, reinterpret_cast<sockaddr*>(&addr), sizeof(addr));
16    listen(server_fd, /*backlog=*/5);
17    return server_fd;
18 }
19
20 void run_iterative_loop(int server_fd) {
21     while (true) {
22         sockaddr_in client_addr{};
23         socklen_t len = sizeof(client_addr);
24         int client_fd = accept(server_fd,
25                                reinterpret_cast<sockaddr*>(&
26                                client_addr), &len);
27         if (client_fd < 0) continue;
28
29         parse_incoming_stream(client_fd); // REQ-PRT-203, see Section
30         close(client_fd);                // REQ-NET-102: close before
31     }
32 }
```

Note that `run_iterative_loop` never returns and never calls `exit()`. A malformed or unsupported packet (Section 6) must degrade gracefully back to this loop, not terminate the process.

2.2 Client Socket Contract (REQ-NET-103)

The client establishes a single active connection to a specified IPv4 address (loopback or remote) and port, transmits exactly one packet, and reads the connection to completion.

```
1 #include <arpa/inet.h>
2
3 int connect_to_server(const char* ip_address, uint16_t port) {
4     int sock_fd = socket(AF_INET, SOCK_STREAM, 0);
5
6     sockaddr_in server_addr{};
7     server_addr.sin_family = AF_INET;
8     server_addr.sin_port = htons(port);
```

```

9     inet_pton(AF_INET, ip_address, &server_addr.sin_addr);
10
11     if (connect(sock_fd, reinterpret_cast<sockaddr*>(&server_addr),
12             sizeof(server_addr)) < 0) {
13         return -1; // Caller decides how to report a failed connection
14     }
15     return sock_fd;
16 }

```

2.3 Byte-Order Normalization (REQ-NET-104)

All multi-byte integer fields (`key_length`, `payload_len`) must be normalized to Network Byte Order (Big-Endian) with `htons()` before transmission, and restored to Host Byte Order with `ntohs()` immediately upon ingestion. Section 3.2 defines the paired `serialize_header` / `deserialize_header` functions that own this responsibility exclusively—no other subsystem may call `htons()` or `ntohs()` directly.

3 Protocol Structures & Serialization (REQ-PRT)

The network boundary requires deterministic memory layouts. The PCSP enforces a strict 6-byte header (REQ-PRT-201). However, compilers naturally align structures to word boundaries, silently inserting padding bytes. We must instruct the compiler to pack the structure contiguously.

3.1 Header Structure Definition

```

1 #include <cstdint>
2
3 #pragma pack(push, 1) // Disable compiler padding
4 struct PCSPHeader {
5     uint8_t version;           // Protocol version, fixed at 0x01
6     uint8_t cipher_id;        // Hexadecimal mapping to cipher
7     uint16_t key_length;      // Length of inline key (Network Byte Order)
8     uint16_t payload_len;     // Length of payload (Network Byte Order)
9 };
10 #pragma pack(pop)           // Restore default padding

```

3.2 Serialization Contract

The developer implementing the transmission loop must not send the native structure directly if running on a Little-Endian machine, and the developer implementing the receive loop must restore host order before reading any length field.

```

1 #include <arpa/inet.h>
2
3 PCSPHeader serialize_header(const PCSPHeader& host_header) {
4     PCSPHeader net_header;
5     net_header.version      = host_header.version;
6     net_header.cipher_id    = host_header.cipher_id;
7     // Network Byte Order mapping
8     net_header.key_length   = htons(host_header.key_length);

```

```

9     net_header.payload_len = htons(host_header.payload_len);
10     return net_header;
11 }
12
13 // REQ-NET-104: the inverse mapping, required on every ingested header
14 PCSPHeader deserialize_header(const PCSPHeader& net_header) {
15     PCSPHeader host_header;
16     host_header.version      = net_header.version;
17     host_header.cipher_id    = net_header.cipher_id;
18     host_header.key_length   = ntohs(net_header.key_length);
19     host_header.payload_len   = ntohs(net_header.payload_len);
20     return host_header;
21 }

```

3.3 Server Parsing State Machine (REQ-PRT-202, REQ-PRT-203)

The transmission packet is packed contiguously in memory as [Header] [Key Vector] [Payload] (REQ-PRT-202). The server must parse the stream in two distinct phases: first read exactly 6 bytes to recover the header, then dynamically size and read the key and payload fields the header describes (REQ-PRT-203).

```

1 #include <sys/socket.h>
2
3 bool parse_incoming_stream(int client_fd) {
4     // Phase 1: exactly 6 bytes, no more, no less
5     PCSPHeader raw_header{};
6     ssize_t got = recv(client_fd, &raw_header, sizeof(PCSPHeader),
7 MSG_WAITALL);
8     if (got != static_cast<ssize_t>(sizeof(PCSPHeader))) {
9         return false; // Incomplete header; nothing further to parse
10    }
11    PCSPHeader header = deserialize_header(raw_header);
12
13    // Phase 2: dynamic allocation sized directly from the header
14    std::vector<uint8_t> key(header.key_length);
15    if (header.key_length > 0) {
16        recv(client_fd, key.data(), header.key_length, MSG_WAITALL);
17    }
18
19    std::vector<uint8_t> payload(header.payload_len);
20    if (header.payload_len > 0) {
21        recv(client_fd, payload.data(), header.payload_len, MSG_WAITALL);
22    }
23
24    return dispatch_and_report(header, key, payload); // Sections 5.3
    and 4.3
}

```

Note that the state machine only ever reads the exact byte counts the header advertises—it never scans for delimiters, consistent with the null-byte safety rationale in Section 1.

4 Diagnostic & Telemetry Contract (REQ-DSP)

Both endpoints must surface identical, human-readable evidence of what crossed the wire. Isolating this contract from the cryptographic and network layers means the developer writing console output only needs a `std::vector<uint8_t>` and a label—never a cipher object or a socket descriptor.

4.1 Shared Hex-Dump Utility

```
1 #include <iostream>
2 #include <iomanip>
3
4 void print_hex(const std::string& label, const std::vector<uint8_t>&
5 data) {
6     std::cout << label << ": ";
7     for (uint8_t byte : data) {
8         std::cout << std::hex << std::setw(2) << std::setfill('0')
9             << static_cast<int>(byte) << ' ';
10    }
11    std::cout << std::dec << std::endl;
```

4.2 Client-Side Telemetry (REQ-DSP-301)

Prior to transmission, the client must print to standard console: the raw plaintext input, the selected Cipher ID, the key metadata (length and, where meaningful, its literal value), and the resulting ciphertext rendered via `print_hex`.

```
1 void log_client_telemetry(const std::string& plaintext,
2                          uint8_t cipher_id,
3                          const std::vector<uint8_t>& key,
4                          const std::vector<uint8_t>& ciphertext) {
5     std::cout << "Plaintext : " << plaintext << '\n';
6     std::cout << "Cipher ID : 0x" << std::hex << static_cast<int>(
7     cipher_id)
8         << std::dec << '\n';
9     std::cout << "Key Length: " << key.size() << " byte(s)\n";
10    print_hex("Ciphertext", ciphertext);
11 }
```

4.3 Server-Side Telemetry (REQ-DSP-302)

Upon packet receipt, the server must print, in order: the raw unresolved hexadecimal stream as it arrived on the socket, the isolated ciphertext block, and the plaintext string recovered immediately after decryption.

```
1 void log_server_telemetry(const std::vector<uint8_t>& raw_stream,
2                          const std::vector<uint8_t>& ciphertext,
3                          const std::vector<uint8_t>& plaintext) {
4     print_hex("Raw Stream ", raw_stream);
5     print_hex("Ciphertext ", ciphertext);
6     std::cout << "Plaintext : "
7         << std::string(plaintext.begin(), plaintext.end()) << std
8         ::endl;
```

5 The Cryptographic Subsystem API (REQ-CIP)

To ensure that developers assigned to network logic remain completely decoupled from those assigned to cryptography, we introduce a polymorphic interface. The mathematical mapping required is defined universally as $C = \text{Encrypt}(P, K)$ and $P = \text{Decrypt}(C, K)$, applied consistently across every cipher family in REQ-CIP-401 through REQ-CIP-702.

5.1 The ICipher Interface

All cipher implementations (Caesar, Vigenère, Hill, etc.) must inherit from a single pure virtual class. This allows the network parser to factory-generate a cipher based purely on the `cipher_id` byte carried in the header (REQ-PRT-201) and invoke it polymorphically.

```

1 #include <vector>
2 #include <cstdint>
3
4 class ICipher {
5 public:
6     virtual ~ICipher() = default;
7
8     // Encrypts a plaintext vector into a ciphertext vector
9     virtual std::vector<uint8_t> encrypt(
10         const std::vector<uint8_t>& plaintext,
11         const std::vector<uint8_t>& key) const = 0;
12
13     // Decrypts a ciphertext vector back into a plaintext vector
14     virtual std::vector<uint8_t> decrypt(
15         const std::vector<uint8_t>& ciphertext,
16         const std::vector<uint8_t>& key) const = 0;
17 };

```

5.2 Cipher ID Registry

The full registry below is the authoritative mapping from wire-level `cipher_id` to implementation and key format. A key insight for the network/parsing team: *key length alone does not disambiguate a key's internal shape*—a 2-byte key field could be an Affine (a, b) pair or a 2-rail Rail Fence count, so the `cipher_id` must always be resolved first, before the raw key bytes are handed to the cipher.

5.3 The Cipher Factory & Polymorphic Dispatch

A single factory function owns every mapping in the registry above. This is the one function the network/parsing developer calls; it either returns a live `ICipher`, or `nullptr` to signal the unsupported-cipher condition required by REQ-NFR-802 (Section 6.2).

```

1 #include <memory>
2
3 std::unique_ptr<ICipher> create_cipher(uint8_t cipher_id) {
4     switch (cipher_id) {
5         case 0x10: return std::make_unique<CaesarCipher>();

```


ID	Cipher	Family	Key Field Format
0x10	Caesar	Monoalphabetic	1 byte — shift K
0x11	Affine	Monoalphabetic	2 bytes — pair (a, b)
0x12	Atbash	Monoalphabetic	0 bytes — keyless
0x13	Monoalphabetic Substitution	Monoalphabetic	26 bytes — permutation array
0x20	Vigenère	Polyalphabetic	Variable — ASCII string
0x21	Beaufort	Polyalphabetic	Variable — ASCII string
0x22	Autokey	Polyalphabetic	Variable — ASCII priming key
0x30	Playfair	Polygraphic	Variable — keyword string
0x31	Hill	Polygraphic	n^2 bytes — row-major matrix, mod 26
0x40	Rail Fence	Transposition	1 byte — rail count
0x41	Columnar Transposition	Transposition	Variable — keyword string

Table 1: PCSP Cipher ID Registry (REQ-CIP-401 through REQ-CIP-702)

```

6      case 0x11: return std::make_unique<AffineCipher>();
7      case 0x12: return std::make_unique<AtbashCipher>();
8      case 0x13: return std::make_unique<MonoSubstitutionCipher>();
9      case 0x20: return std::make_unique<VigenereCipher>();
10     case 0x21: return std::make_unique<BeaufortCipher>();
11     case 0x22: return std::make_unique<AutokeyCipher>();
12     case 0x30: return std::make_unique<PlayfairCipher>();
13     case 0x31: return std::make_unique<HillCipher>();
14     case 0x40: return std::make_unique<RailFenceCipher>();
15     case 0x41: return std::make_unique<ColumnarTranspositionCipher
>();
16     default: return nullptr; // REQ-NFR-802: unsupported Cipher
ID
17     }
18 }

```

5.4 Implementation Example: Caesar Cipher (REQ-CIP-401)

For a developer assigned to REQ-CIP-401, the task is isolated to implementing the mathematical mapping $C_i = (P_i + K) \pmod{256}$ within the established interface. This example operates over the full $\mathbb{Z}_{256}$ byte domain rather than the restricted alphabet space, per the domain choice permitted in Section 3 of the SRS.

```

1 class CaesarCipher : public ICipher {
2 public:
3     std::vector<uint8_t> encrypt(
4         const std::vector<uint8_t>& plaintext,
5         const std::vector<uint8_t>& key) const override
6     {
7         std::vector<uint8_t> ciphertext;
8         ciphertext.reserve(plaintext.size());
9         uint8_t shift = key.empty() ? 0 : key[0];
10
11         for (uint8_t byte : plaintext) {
12             ciphertext.push_back(static_cast<uint8_t>((byte + shift) %
13             256));
14         }
15         return ciphertext;
16     }
17 }

```

```

15     }
16
17     std::vector<uint8_t> decrypt(
18         const std::vector<uint8_t>& ciphertext,
19         const std::vector<uint8_t>& key) const override
20     {
21         std::vector<uint8_t> plaintext;
22         plaintext.reserve(ciphertext.size());
23         uint8_t shift = key.empty() ? 0 : key[0];
24
25         for (uint8_t byte : ciphertext) {
26             // Modulo arithmetic correction for C++ integer promotion
27             plaintext.push_back(
28                 static_cast<uint8_t>((byte - shift + 256) % 256)
29             );
30         }
31         return plaintext;
32     }
33 };

```

5.5 Implementation Example: Vigenère Cipher (REQ-CIP-501)

Unlike Caesar, the Vigenère key is variable-length and cycles across the plaintext: $C_i = (P_i + K_{i \bmod \text{len}(K)}) \bmod 26$. This example is deliberately restricted to the $\mathbb{Z}_{26}$ alphabet domain named in REQ-CIP-501, and assumes both plaintext and key contain only uppercase ASCII letters (0x41–0x5A); a developer targeting full-byte fidelity would extend the boundary handling explicitly rather than change this contract.

```

1 class VigenereCipher : public ICipher {
2 public:
3     std::vector<uint8_t> encrypt(
4         const std::vector<uint8_t>& plaintext,
5         const std::vector<uint8_t>& key) const override
6     {
7         std::vector<uint8_t> ciphertext;
8         ciphertext.reserve(plaintext.size());
9         if (key.empty()) return plaintext; // No key: identity mapping
10
11         for (size_t i = 0; i < plaintext.size(); ++i) {
12             uint8_t p = plaintext[i] - 'A';
13             uint8_t k = key[i % key.size()] - 'A';
14             ciphertext.push_back(static_cast<uint8_t>((p + k) % 26) + 'A');
15         }
16         return ciphertext;
17     }
18
19     std::vector<uint8_t> decrypt(
20         const std::vector<uint8_t>& ciphertext,
21         const std::vector<uint8_t>& key) const override
22     {
23         std::vector<uint8_t> plaintext;
24         plaintext.reserve(ciphertext.size());
25         if (key.empty()) return ciphertext;
26
27         for (size_t i = 0; i < ciphertext.size(); ++i) {
28             uint8_t c = ciphertext[i] - 'A';

```

```

29         uint8_t k = key[i % key.size()] - 'A';
30         plaintext.push_back(static_cast<uint8_t>((c - k + 26) % 26)
31         + 'A');
32     }
33     return plaintext;
34 }
};

```

5.6 Key Field Interpretation Notes for Remaining Ciphers

The remaining nine ciphers follow the same `ICipher` contract; each developer only needs the key-shape notes below, not the internals of any other cipher.

- **Affine (0x11)** — The 2-byte key field splits into a and b . Decryption requires the modular multiplicative inverse $a^{-1} \pmod{26}$, which only exists when $\gcd(a, 26) = 1$; the implementer should validate this at construction rather than at every character.
- **Atbash (0x12)** — Keyless: $C_i = 25 - P_i$. The key vector is expected to be empty and may be ignored entirely.
- **Monoalphabetic Substitution (0x13)** — The 26-byte key *is* the substitution table itself (a permutation of A–Z), so encryption is a direct array lookup with no arithmetic.
- **Beaufort (0x21)** — $C_i = (K_{i \bmod \text{len}(K)} - P_i) \pmod{26}$. Because subtraction is its own inverse here, `encrypt` and `decrypt` can share one implementation.
- **Autokey (0x22)** — The transmitted key field is only the short priming key; the extended keystream $K_{\text{extended}} = K_{\text{prime}} || P$ is derived at runtime by appending the plaintext (or, on decryption, the plaintext recovered so far) as it is processed.
- **Playfair (0x30)** — The keyword string is used once, at construction, to build a 5×5 matrix; the per-character work is then pure matrix-coordinate lookup with row/column/rectangle rules.
- **Hill (0x31)** — The key bytes are read as an $n \times n$ matrix in row-major order. Decryption requires the matrix inverse mod 26, which does not always exist; see the validation helper in Section 6.2, which the factory or the cipher’s own constructor should invoke before REQ-NFR-802 can be satisfied.
- **Rail Fence (0x40)** — The single key byte is the rail count; the transformation is purely index-based (zig-zag diagonal write, row-by-row read) with no modular arithmetic at all.
- **Columnar Transposition (0x41)** — The keyword string determines both the matrix width and the alphabetical column read-out order; like Playfair, all cipher-specific work happens once at construction.

6 Non-Functional Requirements & Safety (REQ-NFR)

6.1 Memory Constraints (REQ-NFR-801)

The engine must explicitly avoid dynamic stack overflows. Fixed-size C-style buffers and unchecked pointer arithmetic are prohibited; all variable-length data uses `std::vector<uint8_t>` or `std::string`, and any interoperating C code must use bounds-checked calls (`strncpy`, `memcpy` with an explicit length) rather than their unchecked counterparts.

6.2 Graceful Error Recovery (REQ-NFR-802)

If the server detects an unsupported Cipher ID, an un-invertible Hill matrix, or a payload length mismatch, it must reject the operation, print an explicit diagnostic warning code, and safely reset the socket channel *without* terminating the server process. This is why `run_iterative_loop` (Section 2.1) is structured as an unconditional loop: every exit path from `parse_incoming_stream` is a `bool`, never a process exit.

```
1 enum class PCSPStatus : uint8_t {
2     OK = 0x00,
3     ERR_UNSUPPORTED_CIPHER = 0x99,
4     ERR_MATRIX_NOT_INVERTIBLE = 0x9A,
5     ERR_LENGTH_MISMATCH = 0x9B
6 };
7
8 // Extended Euclidean check: a Hill key matrix is invertible mod 26
9 // iff gcd(determinant mod 26, 26) == 1.
10 bool is_invertible_mod26(int determinant) {
11     int d = ((determinant % 26) + 26) % 26;
12     int a = d, b = 26;
13     while (b != 0) {
14         int t = b;
15         b = a % b;
16         a = t;
17     }
18     return a == 1;
19 }
20
21 PCSPStatus validate_packet(const PCSPHeader& header,
22                           size_t key_bytes_read,
23                           size_t payload_bytes_read) {
24     if (key_bytes_read != header.key_length ||
25         payload_bytes_read != header.payload_len) {
26         return PCSPStatus::ERR_LENGTH_MISMATCH;
27     }
28     if (create_cipher(header.cipher_id) == nullptr) {
29         return PCSPStatus::ERR_UNSUPPORTED_CIPHER;
30     }
31     // Hill-specific determinant check is delegated to HillCipher's
32     // own constructor/validator once instantiated (see Section 5.6).
33     return PCSPStatus::OK;
34 }
```

On any non-OK status, the server prints the corresponding diagnostic code to standard output and returns control to `run_iterative_loop`, which simply closes the socket and calls `accept()` again—the process itself never terminates.

7 Division of Labor & Integration

By utilizing this rigid documentation:

1. **Network Operations:** One developer can write the socket bindings (`SOCK_STREAM`) and `recv()` loops without knowing how matrix inversion works for the Hill cipher (REQ-CIP-602).
2. **Cryptographic Implementations:** Other developers can construct isolated `ICipher` derivatives, writing local `main()` functions to mathematically test their modules without touching a single network socket.
3. **State Parsing:** A third developer links the two, reading 6 bytes, evaluating the `cipher_id`, instantiating the correct `ICipher` object via the factory, and passing the `std::vector` payloads between them.
4. **Diagnostics:** A fourth developer implements `print_hex` and the telemetry contract against plain byte vectors and labels only—they need no knowledge of cipher math or socket plumbing to satisfy REQ-DSP-301/302.
5. **Safety & QA:** A fifth developer owns `PCSPStatus` and the validation path. They only need the factory's `nullptr` contract and the length fields already parsed out by the state machine to satisfy REQ-NFR-802, without modifying cipher math or socket code.